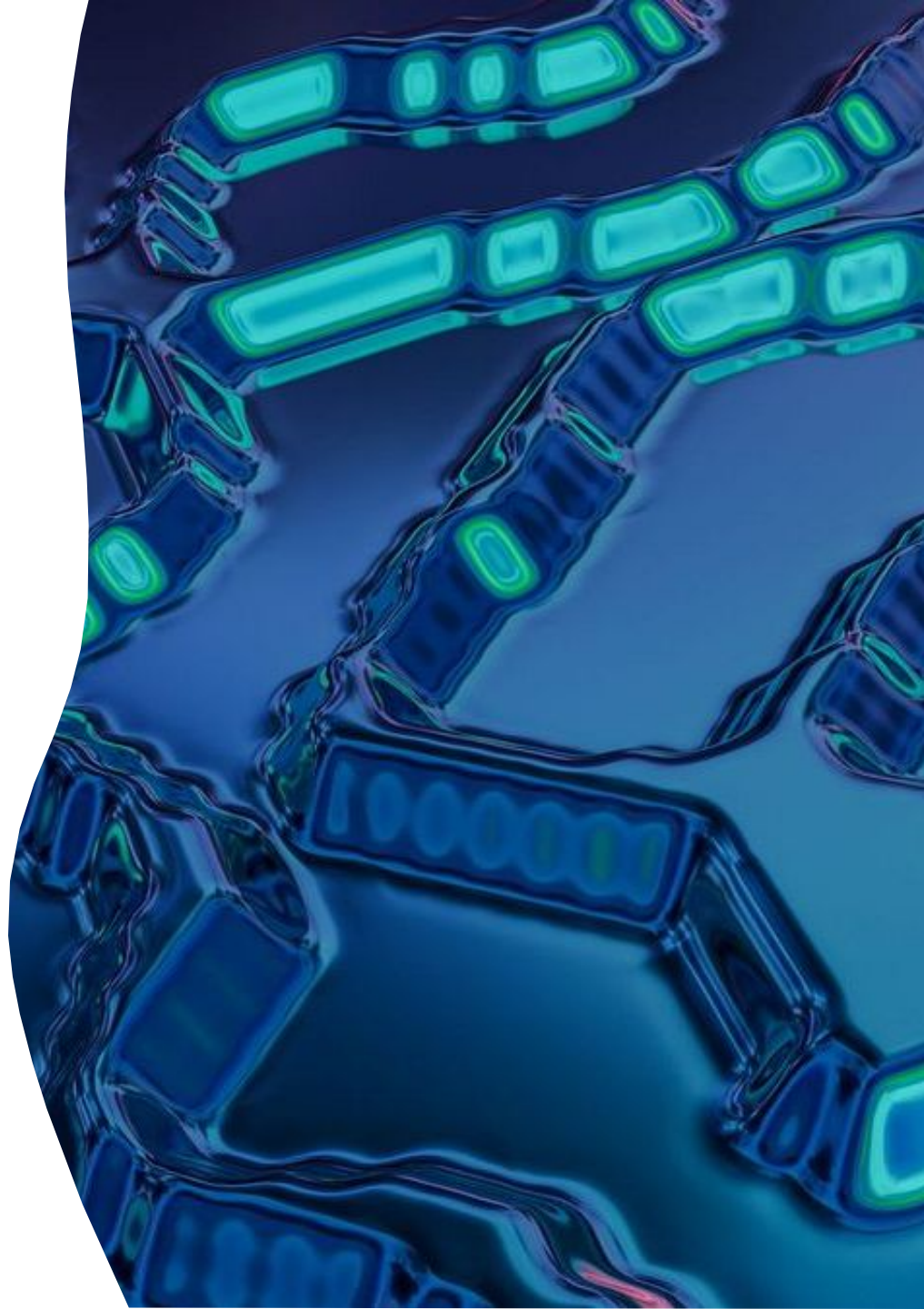
Data Analytics, Data Mining and Machine Learning

Lecture 7

Narges Chinichian · Semester 1, 2026



Overview of this course

Week	Lecture Num	Date	Topic
1	Lecture 1	07.09.2026	Introduction
2	Lecture 2	14.09.2026	Data, Research Questions & Ethics
3	Lecture 3	21.09.2026	Data Visualisation & Exploratory Data Analysis
4	Lecture 4	28.09.2026	Preprocessing & Feature Engineering
5	Lecture 5	05.10.2026	Machine Learning Fundamentals & Regression
6	—	12.10.2026	Project Presentations I
7	Lecture 6	19.10.2026	Classification: Logistic Regression, KNN [Decision Trees is moved to the next session]
8	—	26.10.2026	Bank Holiday — No Class
9	Lecture 7	02.11.2026	Gradient Descent, Regularisation & Model Complexity [+ Decision Trees]
10	Lecture 8	09.11.2026	Model Evaluation & Ensemble Methods
11	Lecture 9	16.11.2026	Neural Networks, Unsupervised Learning & Special Topics
12	—	23.11.2026	Project Presentations II

Quick Quiz

Check Loop



Two Ways to Classify => and now a Third?

	Logistic Regression	KNN
Main idea	Learn a boundary	Look at neighbours
Boundary	Linear (in the features used)	Can be nonlinear
Distance-based?	No	Yes
Scaling important?	Matters with regularisation; helps optimisation	Yes, essential
Main parameter	Regularisation strength	k

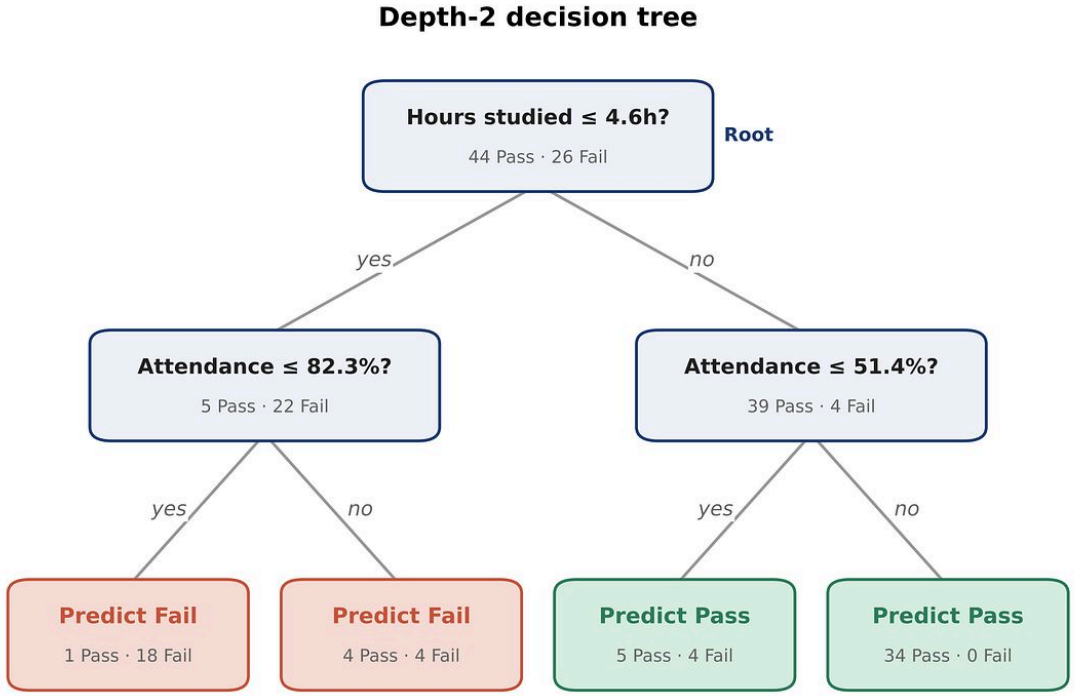
What if we simply ask questions?

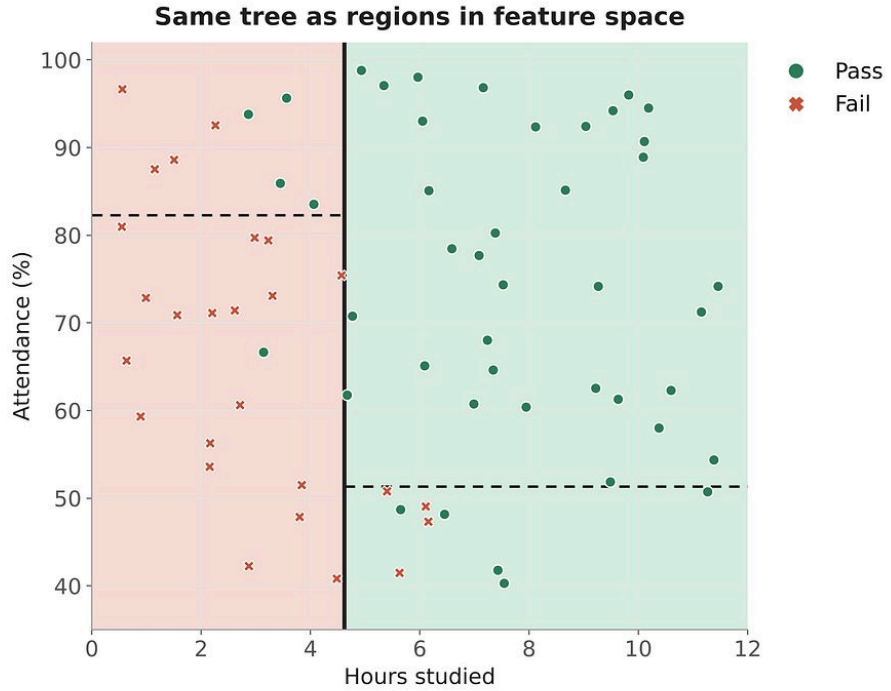
Classification as Questions

Root
the first question

Split
a yes/no question on one feature that divides the data in two

Leaf
the final prediction





Dashed: split makes groups purer, but both sides predict the same class

How Does a Tree Choose Its Questions?

A good split creates **purer** groups: mostly Pass on one side, mostly Fail on the other.

- **Gini impurity / entropy**

  measure how mixed a node is

- **Choose the split with the lowest size-weighted impurity of the children**

- **Repeat on each child: recursive**



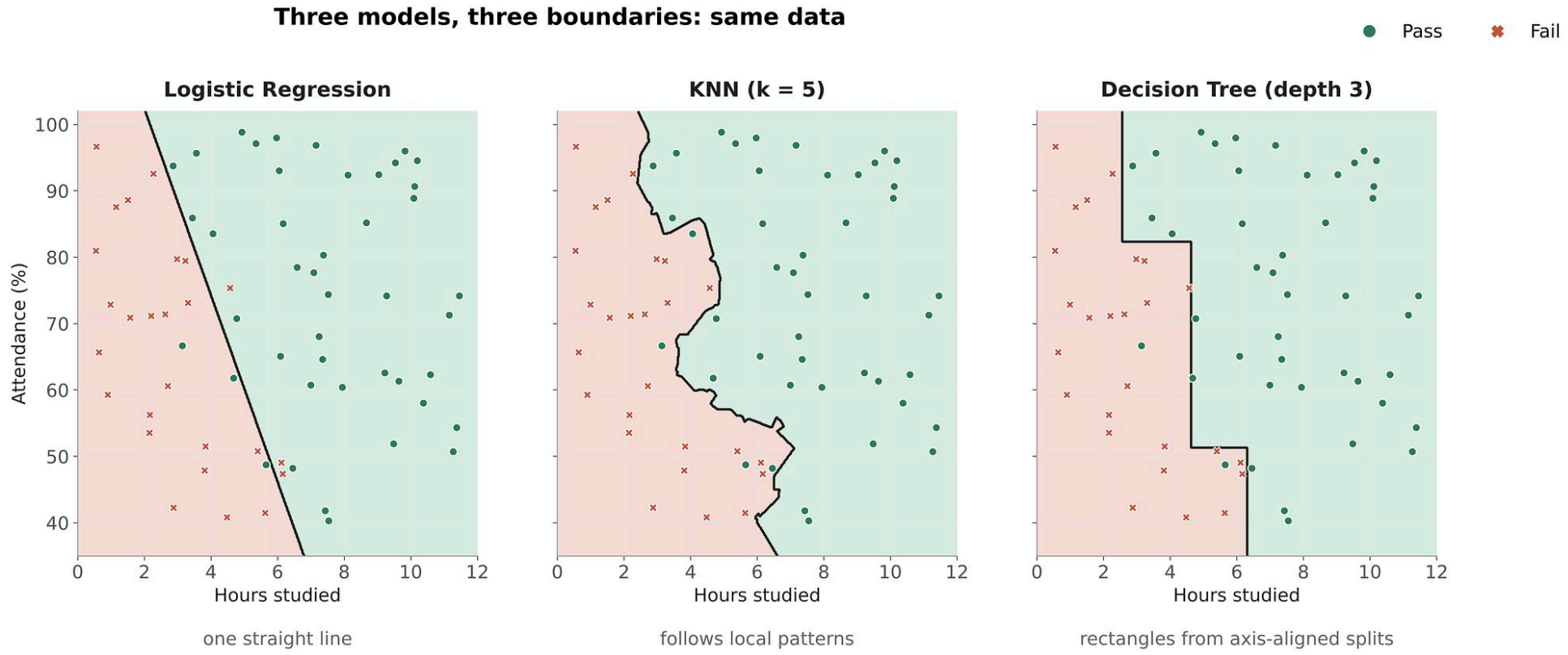
Three Models, Three Boundaries

Logistic Regression
one straight line (with the raw features)

KNN
follows local patterns

Decision Tree
rectangles from axis-aligned splits

Different assumptions → different boundaries.



What Makes Trees Useful?

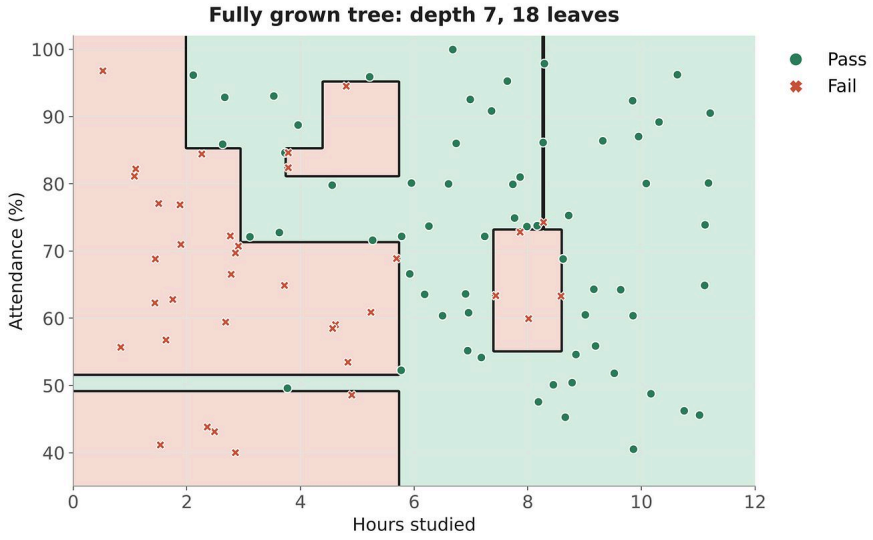
- Readable as rules
- No feature scaling needed
- Captures nonlinearity and interactions

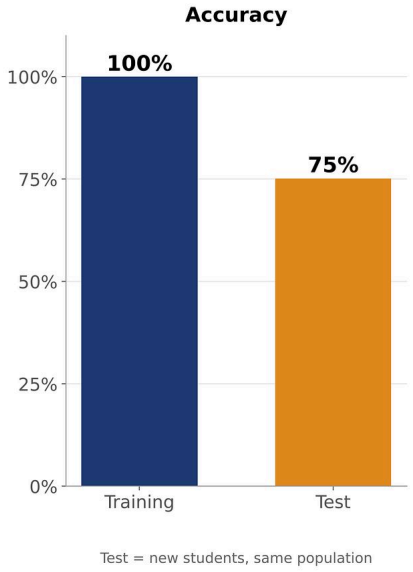
But it is unstable and prone to overfitting when deep.

What if we keep splitting until every leaf is pure?



When a Model Learns Too Much





Training accuracy

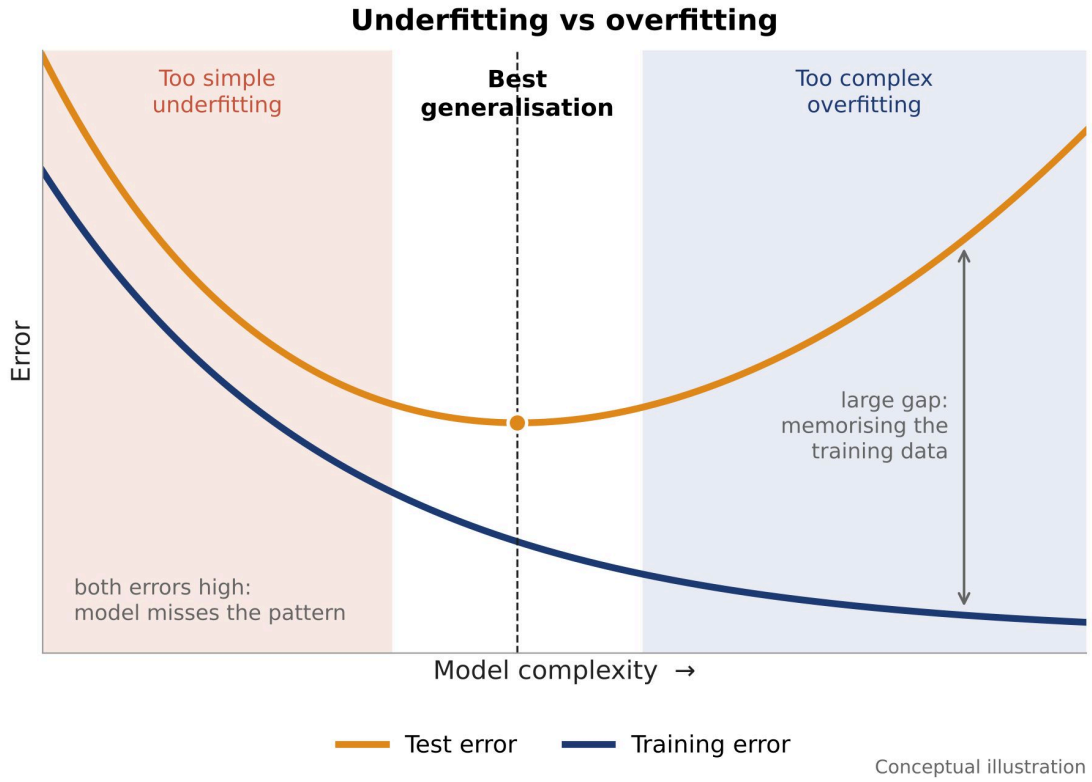
close to 100%

Test accuracy

75%

Fitting seen data ≠ generalising to unseen data.

Underfitting, Overfitting Review



Too simple

underfitting

Too complex

overfitting

In between

best generalisation

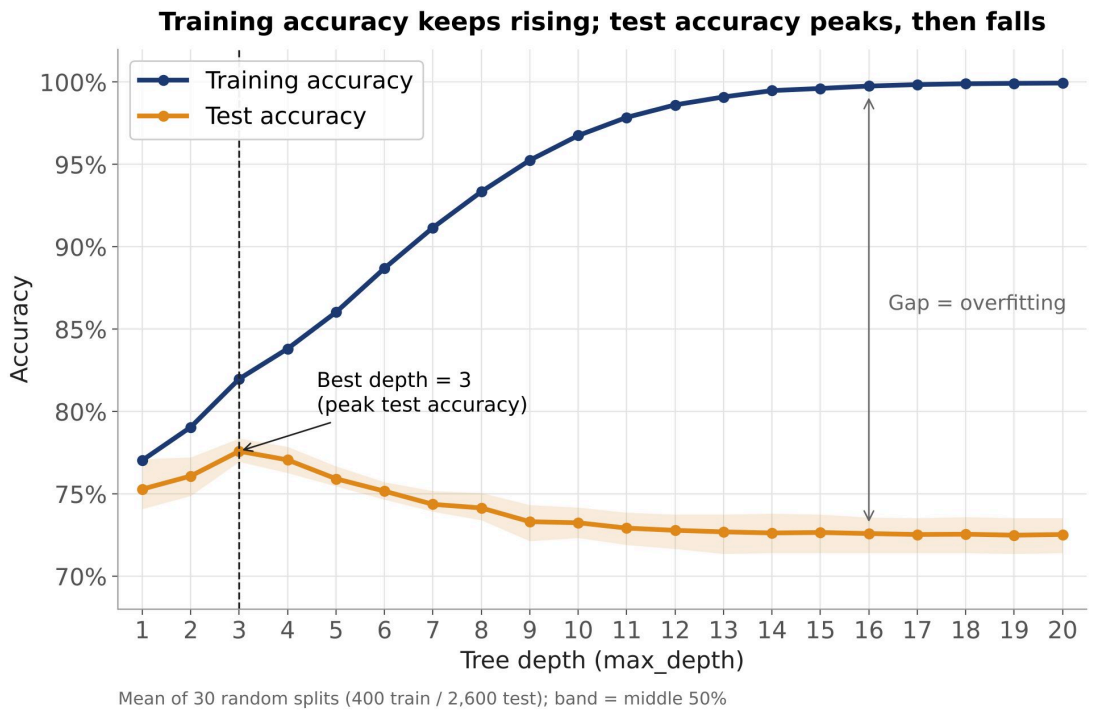
Models Have Different Complexity Knobs:

A few of them we have already learned about:

Model	Knob
Decision Tree	Depth
KNN	k
Polynomial regression	Degree

Choosing the Right Complexity for a Tree

We looked at error; now we look at accuracy:



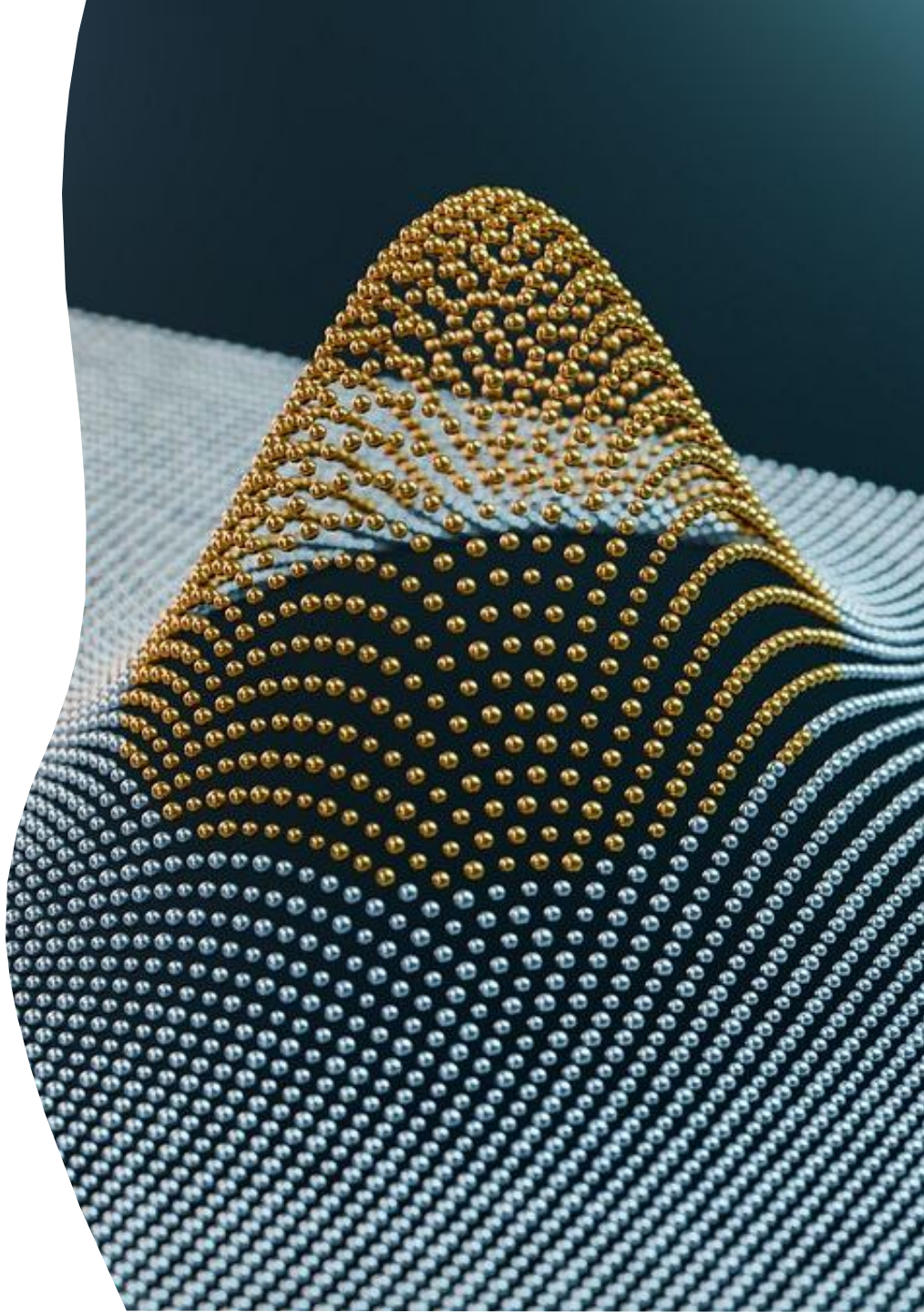
- **Training accuracy keeps rising.**
- **Test accuracy peaks, then falls.**
- **Pick the depth where test accuracy is highest.**

Caveat: choosing with the test set makes it less “unseen”. Normally you’d hold out a separate set for tuning. More in the evaluation lecture.

Regularisation

Techniques that **deliberately discourage** overly complex models.

Instead of the most complex model that fits training data, we add **constraints** or **penalties**.

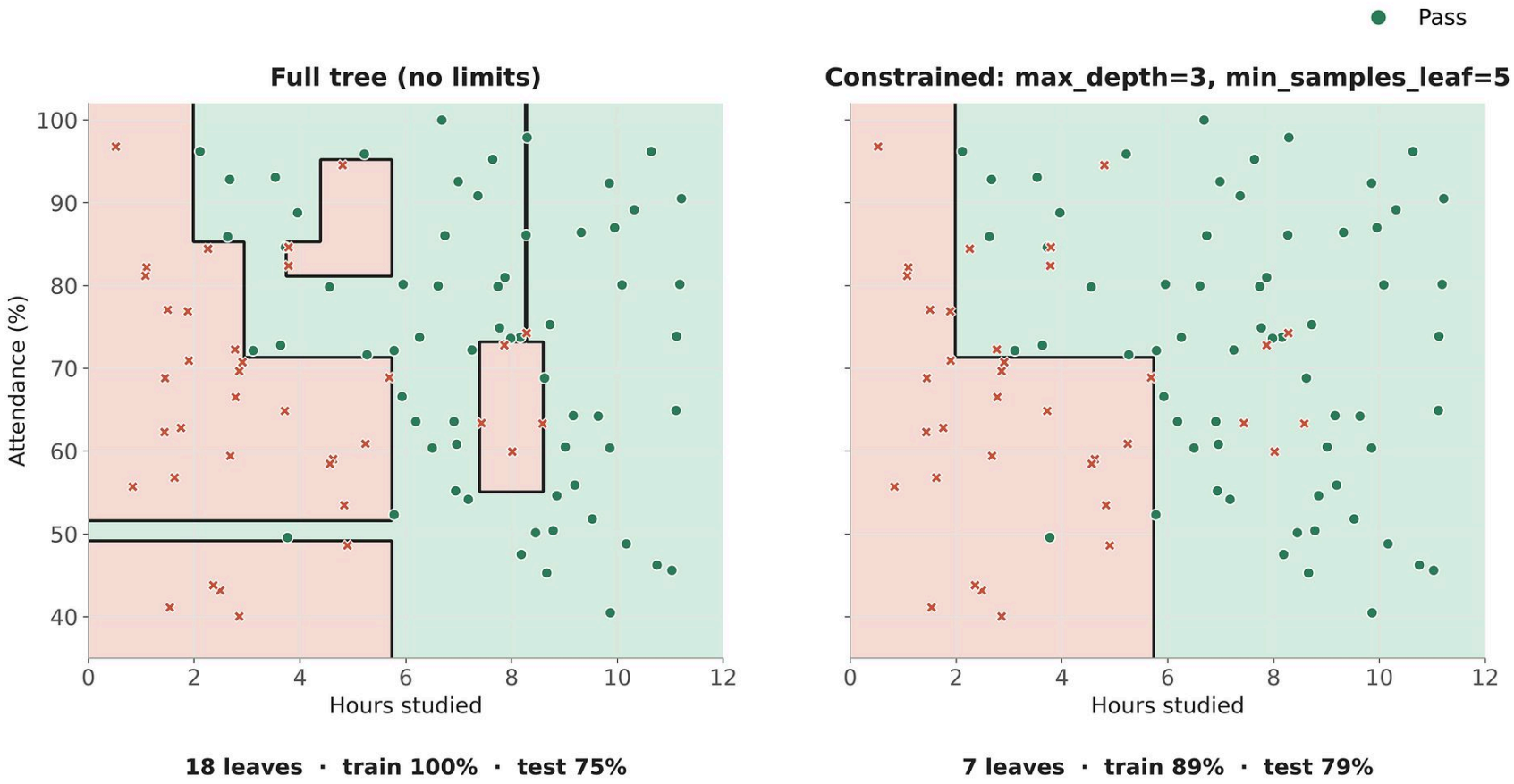


Regularising Decision Trees

max_depth
limit levels

min_samples_leaf
minimum observations per leaf

Pruning
remove branches not worth their complexity



Regularisation: Adding a Price for Complexity

Linear regression so far

Linear regression so far: minimise the error

$$\text{Loss} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad \hat{y} = b_0 + b_1 x_1 + \cdots + b_p x_p$$

The model may use any coefficients that reduce the error, including large ones that just fit noise.

Regularisation

Regularisation: error + a penalty on the coefficients

$$\text{Objective} = \underbrace{\text{Loss}}_{\text{fit the data}} + \lambda \cdot \underbrace{\text{Penalty}(b_1, \ldots, b_p)}_{\text{keep coefficients small}}$$

- **Ridge (L2):** Penalty $= b_1^2 + b_2^2 + \cdots + b_p^2$
- **Lasso (L1):** Penalty $= |b_1| + |b_2| + \cdots + |b_p|$

λ (alpha in scikit-learn) sets the price

- λ = 0 → ordinary linear regression
- λ ↑ → coefficients pushed towards zero → simpler model
- The intercept b_0 is not penalised.

The trade-off: a coefficient only stays large if it reduces the error by more than it costs.

Regularising Linear Models

Objective = Loss + λ · Penalty (λ ↑ → simpler model)

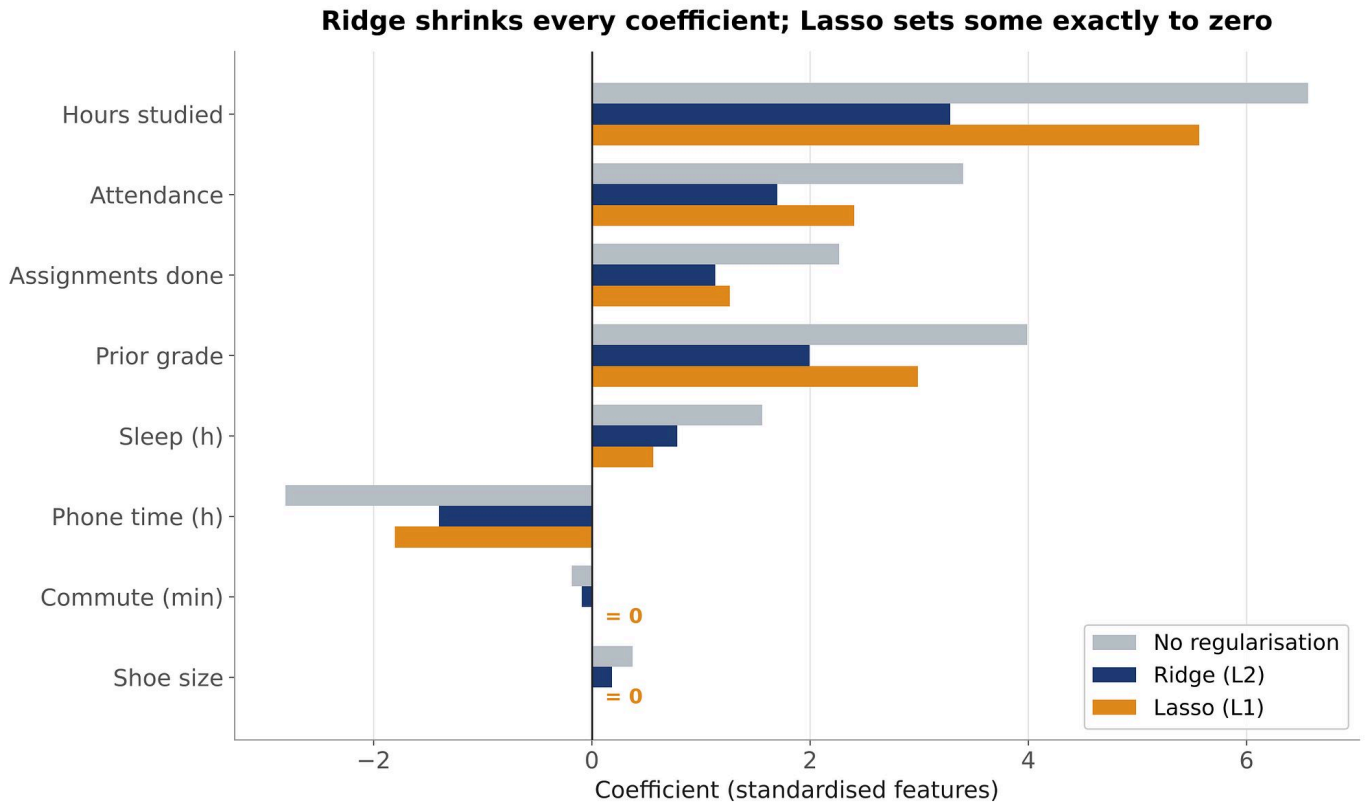
Ridge (L2)

penalty Σb^2; shrinks coefficients towards zero

Lasso (L1)

penalty $\Sigma|b|$; can set coefficients **exactly to zero** (feature selection)

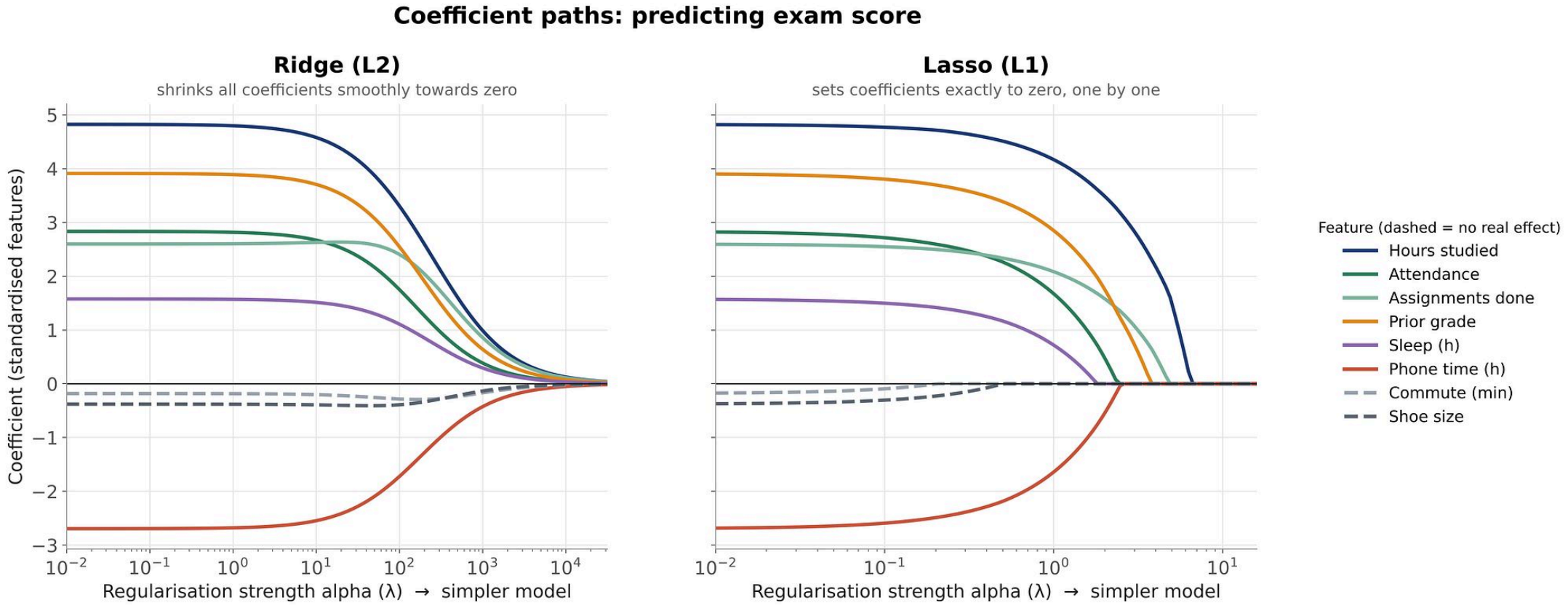
Hoerl & Kennard (1970), Technometrics · Tibshirani (1996), JRSS-B



Predicting exam score. Commute and shoe size have no real effect. Ridge alpha = 200, Lasso alpha = 1.

Regularisation in Practice

scikit-learn: Ridge/Lasso use **alpha** (strength) [What we know as Lambda in our notation]



Ridge vs Lasso coefficient paths

Under the Hood

So far we have:

- **chosen a model** (linear or logistic regression, KNN, decision tree)
- **controlled its complexity** (depth limits, regularisation) so it doesn't overfit

But one step was always hidden in `.fit()`:

How does a model actually find its parameters?

scikit-learn did this for us in every notebook so far.

Now let's see the idea behind it.

How Do Models Find Good Parameters?

Parameters → predictions → loss → adjust

A single tree

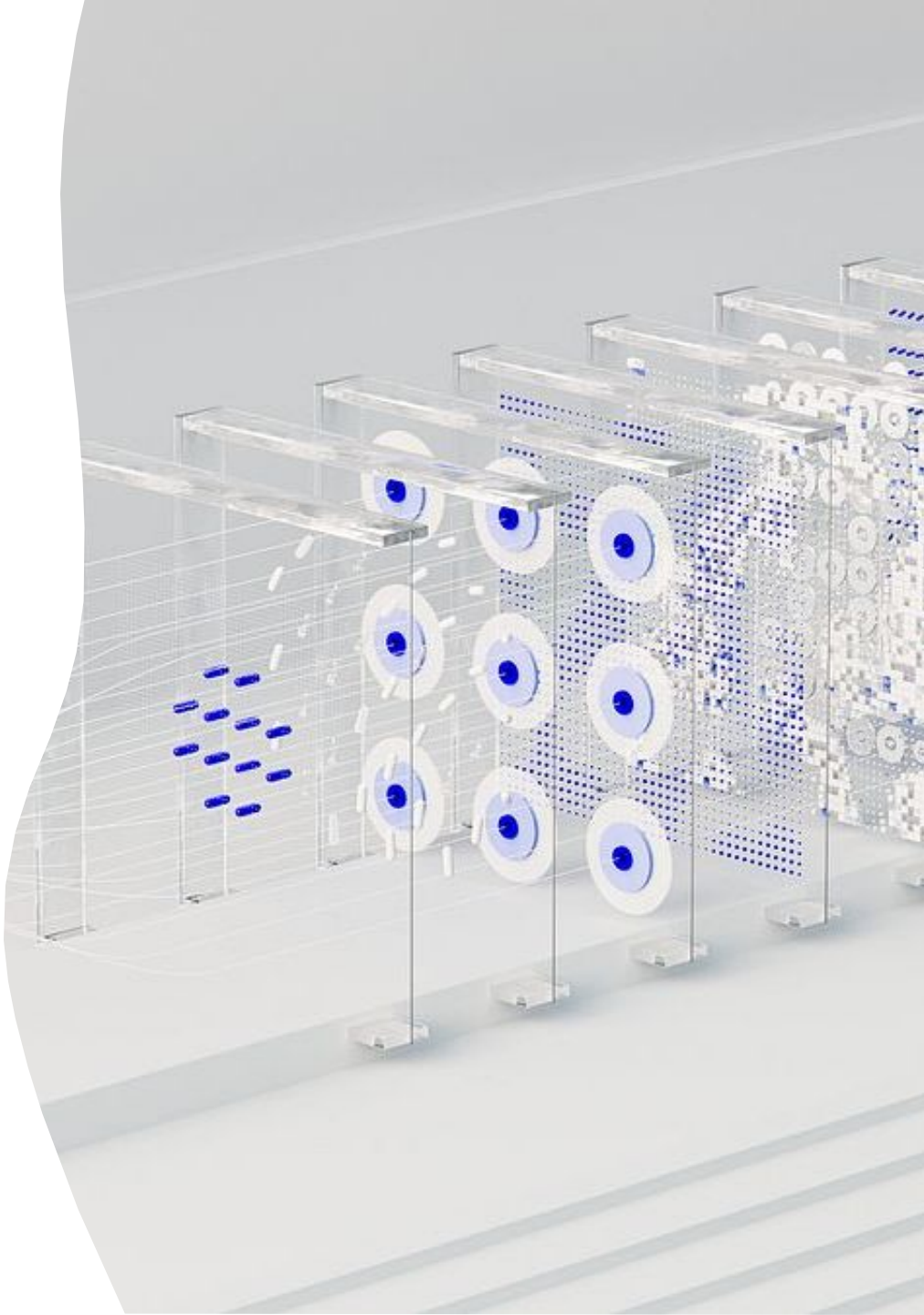
grown greedily, no gradients

Linear regression

closed-form solution (if X^TX is invertible)

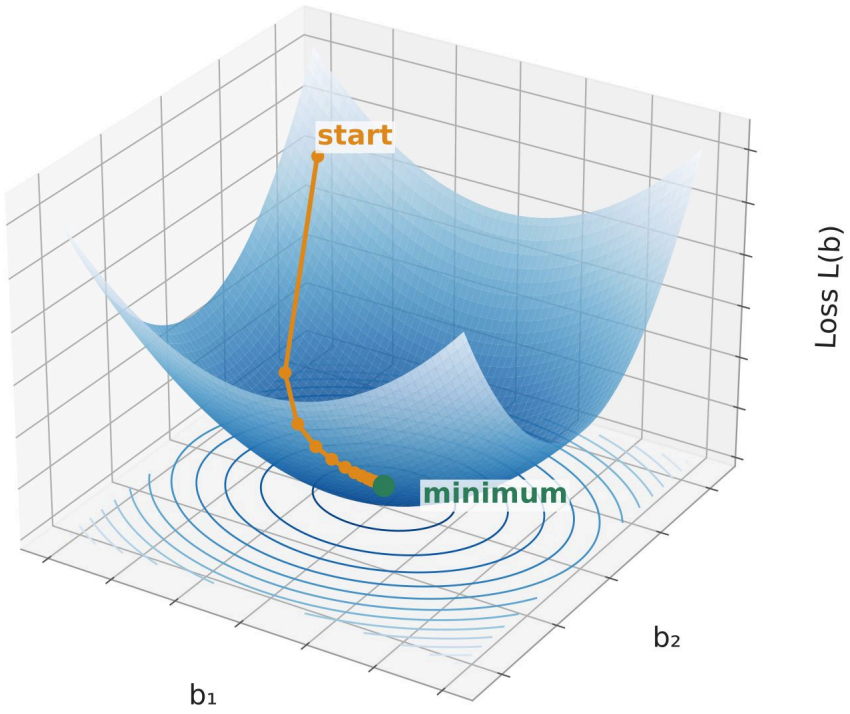
Logistic regression

no closed form → iterative optimisation



The Gradient: Which Way Is Downhill?

Loss surface: each step goes downhill, opposite the gradient



$$b_{new} = b_{old} - \eta \nabla L(b)$$

The gradient points to the steepest **increase** → step the opposite way.

> **$b_{new} = b_{old} - \eta \nabla L(b)$**
>
> η = learning rate

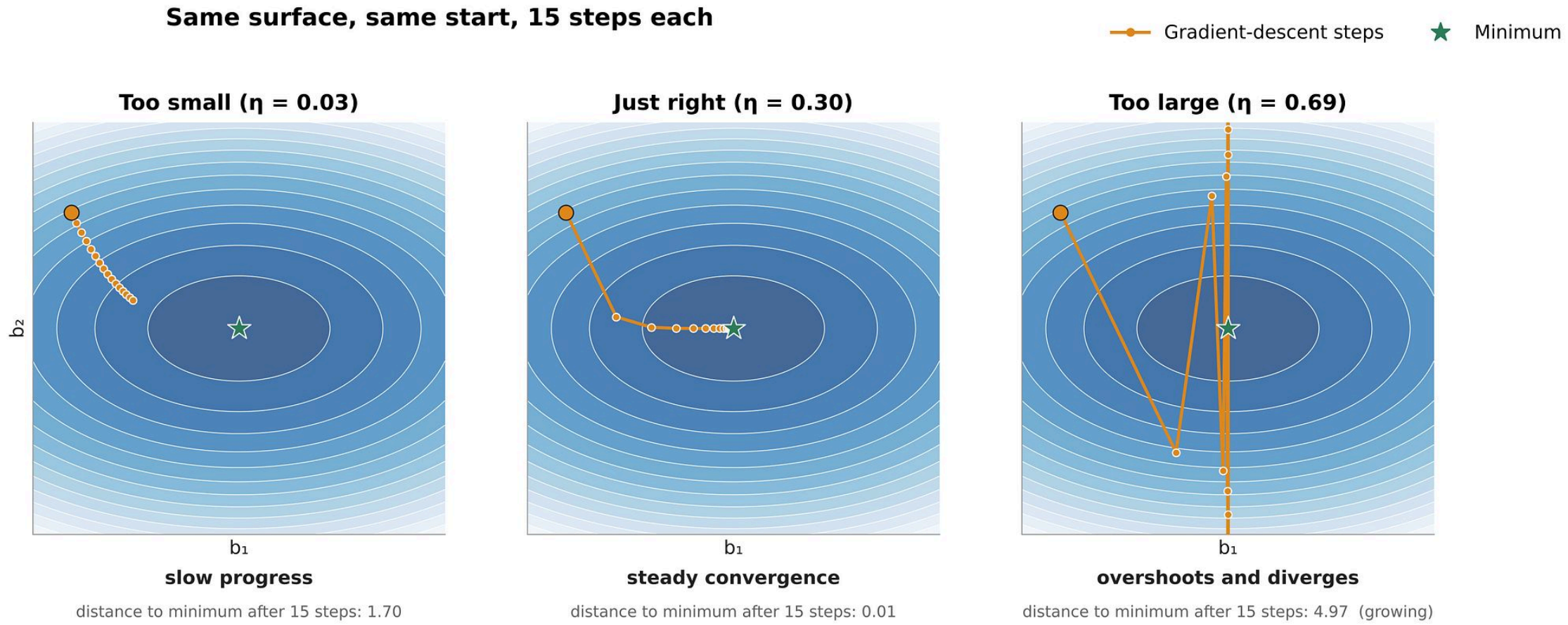
Cauchy (1847) · Goodfellow, Bengio & Courville (2016), Deep Learning §4.3

Learning Rate: How Big a Step?

Too small: slow progress

Just right: steady convergence

Too large: overshoot or diverge



Logistic Regression vs KNN vs Decision Tree

	Logistic Regression	KNN	Decision Tree
Main idea	Learn a boundary	Look at neighbours	Ask a sequence of questions
Boundary	Linear (in the features used)	Can be nonlinear	Can be nonlinear: axis-aligned rectangles
Distance-based?	No	Yes	No
Scaling important?	Matters with regularisation; helps optimisation	Yes, essential	No, splits only compare thresholds
Main parameter	Regularisation strength (`C`)	k	`max_depth`

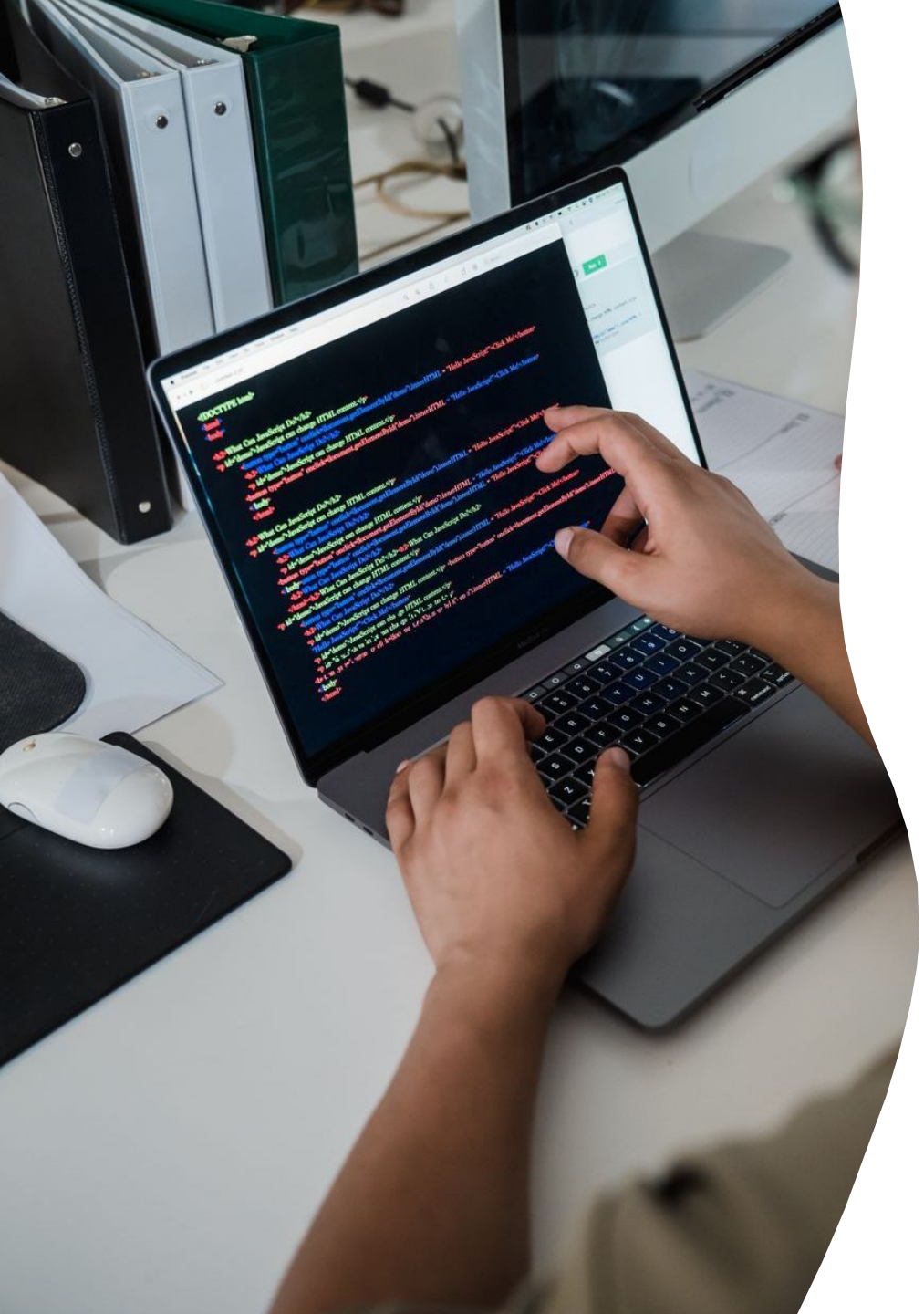
Same problem. Different assumptions.

Key Takeaways

- Trees classify through sequential rules
- Regularisation controls complexity
- Gradient descent iteratively optimises parameters

Next: How do we evaluate fairly, and can many unstable trees beat one?





Hands-On: Investigate the Trade-offs

01

Core

trees with depth 1–20 → plot train vs test accuracy

02

Extension

Ridge & Lasso → vary alpha, plot coefficient paths